

# Code review — Variante BAC

Runde de review pe proiectul tău, cea mai nouă sus.

---

## Runda 2 — 2026-09-23 — sha 7796154

Branch `master`, curat. De la runda 1 ai adăugat patru variante noi (5–8), două fișiere ajutătoare și ai rezolvat trei cerințe care lipseau. 1137 de linii noi.

**Poarta de compilare:** `main.cpp` compilează — dar include un singur fișier ( `temaVar8.h` ) și are `main()` **complet gol**. Verificat cu `g++ -std=c++17 -Wall -fsyntax-only main.cpp`.

**Verificat fișier cu fișier**, fiecare inclus separat:

| Fișier            | Compilează singur                                                                                                                     |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| temaVar1.h        | ✓                                                                                                                                     |
| temaVar2.h        | ✓ (cu warning)                                                                                                                        |
| temaVar3.h        | ✗ ghilimele tipografice + <code>cif</code> definit de două ori                                                                        |
| temaVar4.h        | ✗ ghilimele tipografice + lipsă <code>#include &lt;iostream&gt;</code>                                                                |
| temaVar5.h        | ✗ <code>n</code> , <code>m</code> , <code>A</code> nedecarate + <code>intj</code> + <code>&lt;</code> în loc de <code>&lt;&lt;</code> |
| temaVar6.h        | ✓ (cu warning)                                                                                                                        |
| temaVar7.h        | ✓ (cu warning)                                                                                                                        |
| temaVar8.h        | ✓                                                                                                                                     |
| algortimi-utili.h | ✗ parametri redeclarați în corpul funcției                                                                                            |
| coreactareVar2.h  | ✓ (e gol)                                                                                                                             |

**Patru din zece fișiere nu compilează.** Ca să pot rula ce era de rulat, am lucrat pe o copie în afara repo-ului tău, în care am reparat DOAR liniile cu ghilimele. Codul tău n-a fost atins.

## Întâi, ce ai închis

| Din runda 1                                               | Stare                                              |
|-----------------------------------------------------------|----------------------------------------------------|
| B2 — <code>rezolvare5</code> , vocalele dublate           | ✓ logica e corectă acum                            |
| B4 — <code>bac.txt</code> , numere divizibile cu n        | ✓ rulat: <code>100 40 70 80</code> , exact enunțul |
| B4 — <code>NR.TXT</code> , cel mult 2 cifre, descrescător | ✓ rezolvat                                         |

Și din variantele noi, verificate prin rulare:

| Ce                                         | Rezultat                          |
|--------------------------------------------|-----------------------------------|
| V3 item 5 — matricea pe coloane, n=4       | rulat: exact matricea din enunț ✓ |
| V3 item 3 — cel puțin 3 cifre din NR.TXT   | rulat: 3798 ✓                     |
| V6 item 5 — prima literă a fiecărui cuvânt | rulat: Mare Frig Rosu ✓           |
| V6 item 3 — sumele parțiale                | rulat: 15 / 11 / 8 / 1 ✓          |
| V8 item 3 — pătrate perfecte din BAC.TXT   | rulat: 9+36+9=54 ✓                |
| V5 item 3 — prefixele lui n                | rulat: 10305 1030 103 10 1 ✓      |
| V5 item 4a — cel mai mic divizor prim      | rulat: 45→3, 4→2, 101→101 ✓       |
| V8 pseudocod, n=932125, k=3                | ai scris 22 · rulat: 22 ✓         |
| V7 SUB3 — f(12345)                         | ai scris 42 · rulat: 42 ✓         |
| V6, V7 — itemii cu variante de răspuns     | corecți                           |

transformaVar6 merge inclusiv pe cuvinte separate prin **mai multe** spații — condiția `s[i-1]==' '` && `s[i]!=' '` e exact cea bună. Ai ales-o singur.

Nouă exerciții noi rulate, toate corecte. Problemele de mai jos sunt aproape toate de **altă natură**: fișiere pe care nu le compilezi, răspunsuri pe care nu le scrii, și cod copiat dintr-un fișier în altul.

## Critice

### B1 — un buletin de analize medicale e public pe GitHub

buletinAnalyze\_2010.pdf , comis în 7796154 , repo dariusswts/Variante-BAC , vizibilitate **PUBLIC**. 3 pagini, titlul documentului: BULETIN ANALIZE .

### CUM E ACUM

Fișierul e urmărit de git și pushat. Oricine deschide linkul repo-ului îl poate descărca. N-am deschis conținutul.

## DE CE

Aici nu e vorba de programare, dar e lucrul cel mai urgent din toată runda. Probabil a intrat cu un `git add .` — comanda aia adaugă tot ce e în folder, inclusiv ce ai pus acolo din alt motiv și ai uitat. De asta `.gitignore` nu e o formalitate: e lista lucrurilor pe care `git add .` are voie să le sară.

Partea care contează cel mai mult: **ștergerea fișierului și un commit nou NU îl scot din istoric**. Git păstrează fiecare versiune a fiecărui fișier. După `git rm` + commit, oricine face `git clone` ia și obiectul vechi, iar linkul direct către commit-ul `7796154` continuă să funcționeze. Un fișier ajuns într-un repo public trebuie tratat ca deja copiat.

## FIX

Întâi fă repo-ul privat, din Settings → General → Change visibility. Asta oprește hemoragia în treizeci de secunde. Abia apoi curățăm istoricul — nu o face singur, vorbim înainte, pentru că rescrierea istoricului strică toate clonele existente.

Și adaugă un `.gitignore` înainte de următorul `git add .`.

---

**B2 — `temaVar3.h:63,67` și `temaVar4.h:82,87` : aceleași patru linii, neatinse**

## CUM E ACUM

Identice cu runda 1, caracter cu caracter:

```
cout<<'x'; | printf("%c",'x');
```

```
./temaVar3.h:63:15: error: unexpected character <U+2019>
./temaVar4.h:82:21: error: unexpected character <U+201D>
```

## DE CE

Explicația e în runda 1 și rămâne valabilă: bara `|` nu e cod, e notația prin care subiectul îți dă aceeași instrucțiune în C++ și în C; ghilimelele `'` și `"` din PDF nu sunt `'` și `"` de pe tastatură.

Ce e nou și e de fapt constatarea acestei runde: **ai scris 1137 de linii peste un fișier despre care ți se spusese că nu compilează**. Mecanismul e cel de la runda 1 — `main.cpp` nu include fișierele, deci compilatorul nu se uită în ele niciodată. Atât timp cât poarta nu e închisă, fiecare fișier nou pe care îl scrii e încă un fișier necontrolat. Și se vede: `temaVar5.h`, scris **după** review, are exact aceeași greșeală, la linia 72.

## FIX

```
// temaVar3.h:63 și :67
cout << 'x';
cout << 'y';
// temaVar4.h:82 și :87
cout << 0;
cout << 1;
// temaVar5.h:72
cout << '*';
```

Apoi include toate fișierele în `main.cpp` și compilează. Vei primi erorile de la B3 și B4 — e normal, și e exact motivul pentru care pasul ăsta e primul.

---

**B3 — `temaVar3.h:232 : cif` e definit de două ori în același fișier**

## CUM E ACUM

```
./temaVar3.h:232:5: error: redefinition of 'cif'
./temaVar3.h:101:5: note: previous definition is here
```

Linia 101 și linia 232 sunt aceeași funcție, cu aceeași semnătură, cu corpul identic în afară de numele variabilei ( `cnt` vs `nr` ). Și enunțul de deasupra e copiat de două ori: liniile 87–99 și 226–230.

## DE CE

În C++, o funcție poate fi **declarată** de câte ori vrei, dar **definită** o singură dată. Definiția e locul unde generează compilatorul cod mașină; două definiții înseamnă două bucăți de cod pentru același nume, și n-are cum să aleagă. De aia mesajul e `redefinition`, nu „ambiguitate”.

Observă că a doua funcție nu e greșită. E corectă. Dar existența ei blochează complet compilarea fișierului — deci și prima, cea bună, devine inutilizabilă. Duplicarea nu e „cod în plus care nu

deranjează", e cod care oprește totul.

## FIX

Șterge liniile 226–244 (al doilea enunț și a doua definiție). Prima, de la linia 101, e cea pe care o păstrezi.

## B4 — cinci funcții definite în două fișiere diferite

### CUM E ACUM

Am inclus toate cele opt fișiere împreună, după ce reparasem ghilimelele:

```
./temaVar4.h:124: error: redefinition of 'rezol5'
    ./temaVar3.h:154: note: previous definition is here
./temaVar5.h:93: error: redefinition of 'rezolvare3'
    ./temaVar3.h:212: note: previous definition is here
./temaVar5.h:70: error: redefinition of 'f'
    ./temaVar4.h:80: note: previous definition is here
./temaVar7.h:42: error: redefinition of 'transformaVar6'
    ./temaVar6.h:78: note: previous definition is here
./temaVar3.h:59: error: functions that differ only in their return type
                cannot be overloaded
    ./temaVar2.h:196: note: previous definition is here
```

### DE CE

Gărzile de includere ( `#ifndef TEMAVAR3_H_INCLUDED` ) pe care le ai pun fiecare fișier o singură dată — asta funcționează. Dar ele apară împotriva includerii **aceluiași** fișier de două ori, nu împotriva a **două fișiere diferite** care definesc același nume. Pentru compilator, după ce le lipește pe toate, există un singur spațiu de nume global, și acolo `rezol5` apare de două ori.

Ultima eroare e altfel și merită privită separat: `int f(int a)` din `temaVar2.h` și `void f(int x)` din `temaVar3.h` au **parametri identici** și diferă doar prin tipul returnat. C++ alege funcția după parametri, niciodată după ce returnează — la `f(5)` n-ar avea nicio informație pe baza căreia să decidă. De aia asta nu e nici măcar supraîncărcare validă.

Ce se vede în spatele tuturor: `temaVar7.h` conține copia exactă a lui `transformaVar6` și `rezVar6` din `temaVar6.h`, iar `temaVar4.h` conține copia exactă a lui `rezol5` din `temaVar3.h` — cu tot cu

enunțul variantei 3, lipit la liniile 109–122. Ai copiat fișierul, nu soluția.

## FIX

Un nume, un fișier. `transformaVar6` rămâne doar în `temaVar6.h`; `temaVar7.h` îl folosește printr-un `#include "temaVar6.h"`. La fel pentru `rezol5`. Iar funcțiile de urmărire ale subiectului III redenumesc-le după variantă — `f_var4`, `f_var5` — pentru că sunt funcții diferite care se cheamă la fel doar pentru că așa apar în subiect.

---

## B5 — `temaVar4.h:124` : ai rezolvat cerința altei variante

### CUM E ACUM

Enunțul variantei 4, de la linia 53: diagonala **secundară** primește 0, prima **linie** primește  $n$ , a doua **linie**  $n-1$ . Exemplul din enunț, pentru  $n=4$ :

```
4 4 4 0
3 3 0 3
2 0 2 2
0 1 1 1
```

Rulat `rezol5()` din `temaVar4.h`, cu  $n=4$ :

```
0 3 2 1
4 0 2 1
4 3 0 1
4 3 2 0
```

### DE CE

Matricea pe care o produci e corectă — pentru varianta 3. Acolo cerința e pe diagonala **principală** și pe **coloane**, iar `a[i][j] = n-j` e exact ce trebuie. În varianta 4 cerința e întoarsă pe amândouă axe: diagonala secundară înseamnă `i+j == n-1`, nu `i==j`, iar valoarea depinde de **linie**, nu de coloană, deci `n-i`, nu `n-j`.

Ce s-a întâmplat de fapt: ai lipit în `temaVar4.h`, la liniile 109–122, enunțul variantei 3 — a doua oară în același fișier, sub enunțul corect care era deja acolo la linia 53. Apoi ai rezolvat enunțul de jos. Nu e o greșeală de gândire, e o greșeală de citit: ai rezolvat ce aveai sub ochi, nu ce cerea subiectul.

La bac asta se punctează cu 0 chiar dacă programul e impecabil.

## FIX

```
void rezol5Var4(int n){
    int a[25][25];
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            if(i+j==n-1)
                a[i][j]=0;
            else
                a[i][j]=n-i;
    // afișarea rămâne cum e
}
```

Și șterge enunțul duplicat de la liniile 109–122, ca să nu te mai încurce.

## B6 — temaVar2.h:75 : șirul afișat nu e terminat

### CUM E ACUM

Rulat `rezolvare5()` pe `bacalaureat` :

```
baAcaAlaAuUreEaAt\M-^@k^A
```

Enunțul cere `baAcaAlaAuUreEaAt` . Literele sunt corecte — după ele apar trei octeți de gunoi.

### DE CE

Aici e mecanismul care te va urmări la fiecare exercițiu cu șiruri, deci merită înțeles o dată bine.

Un `char[100]` nu știe câte caractere are. Sunt o sută de octeți, atât. Ce marchează sfârșitul textului e un octet cu valoarea 0 — `'\0'` — pus **în interiorul** tabloului. `cout << cuvântNou` pornește de la primul octet și scrie până dă de el.

`cuvântNou` e o variabilă locală, deci conține ce era pe stivă înainte — resturi de la alte apeluri. Tu scrii 17 caractere în ea și te oprești. `cout` scrie cele 17 caractere, ajunge la octetul 18, care nu e zero, și

continuă. Se oprește abia când nimerește din întâmplare un zero. De aia gunoiul e de lungime variabilă și de aia programul pare că „merge uneori”.

Compară cu `cuvant`: acolo `cin >> cuvant` pune el `'\0'` la final. De aia bucelele tale pe `cuvant` funcționează. Când construiești tu șirul, terminatorul e treaba ta.

## FIX

```
cuvantNou[poz] = '\0';  
cout << cuvantNou << endl;
```

O singură linie, înainte de afișare.

---

## B7 — `temaVar5.h:139–162`: funcțiile lucrează pe variabile care nu există

### CUM E ACUM

```
./temaVar5.h:139:21: error: use of undeclared identifier 'n'  
./temaVar5.h:144:17: error: use of undeclared identifier 'A'  
./temaVar5.h:157:14: error: use of undeclared identifier 'intj'  
./temaVar5.h:158:19: error: use of undeclared identifier 'A'
```

## DE CE

`construireVar5()` și `afisareVar5()` sunt scrise ca și cum `A`, `n` și `m` ar fi vizibile de oriunde. Dar ele sunt declarate la linia 166, **înăuntrul** lui `rezolvareVar5()`. O variabilă declarată într-o funcție trăiește doar acolo: se naște când intri în funcție, moare când ieși, și nicio altă funcție nu o vede. Compilatorul nici măcar nu o caută — pentru el, în `construireVar5`, numele `A` n-a fost declarat niciodată.

Două funcții care trebuie să lucreze pe aceeași matrice trebuie să și-o **paseze**, ca parametru. Exact cum ai făcut corect în `temaVar3.h`, la `citireVar3(v,n)` și `sortareVar3(v,n)`. Știi tiparul — aici nu l-ai aplicat.

`intj=1` de la linia 157 e o tastă lipsă (`int j`), iar `cout<<A[i][j]< " "` de la 158 are un singur `<` în loc de două — a doua e comparație, nu afișare.

## FIX

```

void construireVar5(int A[][11], int n, int m){
    for(int i=1;i<=n;i++)
        for(int j=1;j<=m;j++)
            A[i][j] = (i>j) ? i : j;
}

void afisareVar5(int A[][11], int n, int m){
    for(int i=1;i<=n;i++){
        for(int j=1;j<=m;j++)
            cout << A[i][j] << " ";
        cout << endl;
    }
}

void rezolvareVar5(){
    int A[11][11], n, m;
    cin >> n >> m;
    construireVar5(A, n, m);
    afisareVar5(A, n, m);
}

```

**B8 —** `temaVar5.h:81` : `f(3,17)` afișează `7*****` , nu `****`

## CUM E ACUM

Ai scris, la linia 81: `reaspuns ****`

**Rulat:** `7*****` — un `7` , apoi cinci stele.

## DE CE

Tabelul tău e corect până la capăt: `f(3,17) → f(4,15) → f(5,13) → f(6,11) → f(7,9) → f(8,7)` .  
Sunt cinci apeluri care intră pe ramura `a<=b` , deci cinci stele. Tu ai numărat patru — ai oprit lista la `f(7,9)` fără să pui steaua pentru el.

Dar lipsește și altceva, mai important: apelul `f(8,7)` . Acolo `8<=7` e fals, deci se execută `else` `cout<<b` , care afișează **7**. Și se afișează **primul**, înaintea tuturor stelelor, pentru că e cel mai adânc apel — el termină primul. Stelele se scriu abia la întoarcerea din recursivitate, de la fundul stivei în sus.

Ăsta e exact tiparul din runda 1, la `f(15,2)` : te oprești din urmărire fix la apelul care încheie recursivitatea, și în recursivitate ultimul apel e cel care scrie primul. **Ramura `else` nu e sfârșitul urmăririi, e începutul afișării.**

FIX

```
raspuns: 7*****
```

## 🟡 Importante

**M1 — `main.cpp:12 : main()` e gol.** Include un singur fișier ( `temaVar8.h` ) și nu apelează nimic. E cauza lui B2: opt fișiere, unul singur compilat. Include-le pe toate și apelează, pe rând, soluția la care lucrezi. (Semnalat ca M4 în runda 1, neatins.)

**M2 — `temaVar2.h:171 : n` intră neinițializat în `citire3`.** `int v[100],n;` — `n` nu primește valoare, și `citire3` îl folosește direct ca indice: `while(f>>v[n])` . N-am putut reproduce o rulare greșită (pe mașina mea a nimerit 0 de fiecare dată), dar valoarea e nedefinită prin standard: poate fi orice, inclusiv 5000, caz în care scrii în afara tabloului. Compară cu `citireVar3` din `temaVar3.h:184` , unde ai scris `n=0;` — acolo ai făcut-o corect.

**M3 — `temaVar2.h:130 : citire3` afișează, deși ar trebui doar să citească.** Rulat `rez3()` pe `NR.TXT` , pe ecran apare:

```
7 7 51 6 3798
dupa sortare
6 7 7 51 3798
```

Enunțul cere o singură linie, sortată. Ai trei. `citire3` face și citirea, și filtrarea, și afișarea, și mesajul **NU EXISTA!** — apoi `afisare3` le face pe ultimele două a doua oară. Scoate din `citire3` tot ce ține de afișat: rămâne citirea, care e treaba ei.

**M4 — `temaVar2.h:196 : int f(int a){ }` nu returnează nimic.**

```
./temaVar2.h:198:1: warning: non-void function does not return a value
```

E cerința 4a de la varianta 2 (suma exponenților din descompunerea în factori primi), lăsată goală. Un `int` fără `return` returnează gunoi, iar programul nici nu se plânge la rulare — doar dă rezultate greșite. Același warning la `rezVar6` în `temaVar6.h:96` și `temaVar7.h:59` : sunt declarate `int` dar nu returnează nimic; ar trebui `void` .

**M5 — `algortimi-utili.h` nu compilează, în două locuri.** Linia 50: `int v[100];` redeclară parametrul `v` al funcției — la fel `n` la linia 51. Parametrul este deja o variabilă locală; nu-l poți declara a doua oară. Și linia 9: `int n;` în `alg()` e neinițializat, iar `while(n!=0)` îl citește imediat. În plus, fișierul folosește `cout` fără `#include <iostream>` . Un fișier numit „algoritmi utili” e ultimul loc unde vrei cod care nu merge — acolo te vei uita când ai emoții.

**M6 — `NR.TXT` : un fișier, trei cerințe care cer lucruri diferite.** Varianta 2 vrea numere **întregi** (are negative), varianta 3 și 4 vor numere **naturale**. Fișierul comis are `-3 -10 0 7 -5 7 51 -800 6 3798` — datele variantei 2. Din cauza asta, `rezVar4_3()` afișează `51 7 7 6 0 -3 -5 -10 -800` : filtrul `v[i]<=99` e **corect față de enunț**, dar pe date care încalcă enunțul lasă să treacă tot ce e negativ, și pare stricat când nu e. Peste toate, `rezolvare3()` din `temaVar5.h:97` deschide `NR.TXT` cu `ofstream` și îl **suprascrive** — dacă rulezi varianta 5, datele pentru 2, 3 și 4 dispar. Ține fișiere separate: `nr_var2.txt` , `nr_var3.txt` .

**M7 — `temaVar5.h:32` : răspunsul la pseudocod e 1 , nu `y=532` .** Ai calculat corect prima buclă: `y` ajunge 532. Dar algoritmul mai are două părți, iar întrebarea e ce **afișează**, nu cât e `y` . Urmează bucla care taie simultan ultima cifră din `y` și din `z` cât timp coincid — 532 și 532 coincid cifră cu cifră, deci ambele ajung la 0 — și apoi **dacă `y+z=0` atunci scrie 1** . Rulat: **afișează 1** . Te-ai oprit la variabila intermediară.

Același lucru la `temaVar8.h:61` : ai tabelul pentru `f(3)` , dar niciun rând cu rezultatul. Rulat, `f(3)` afișează `2` și rând nou, apoi `1 2 3` .

---

## Cleanup

**C1 — cerințe rămase doar ca enunț:** `temaVar2.h:89` (ultimul cuvânt generat prin backtracking — răspunsul ți l-am dat în runda 1: `eded` ), `temaVar2.h:191` (extraprim, partea b), `temaVar3.h:92` (cel mai mare palindrom), `temaVar4.h:44` (eliminarea literelor mici), `temaVar4.h:90` ( `f(15,2)` — răspunsul, dat în runda 1: `01111` ), `temaVar7.h:83` (interschimbarea în zone), `temaVar8.h:114` (interclasare cu paritate alternantă). Șapte.

**C2 — `temaVar1.h:9` : litera e tot greșită.** Ai scris condiția corect ( `x>5 si x<=8` ) și ai bifat **b**, care e un **sau** mereu adevărat. Răspunsul e **d**. (B3 în runda 1, zece secunde de lucru.)

**C3 —** `temaVar1.h` **n-are gardă de includere**, spre deosebire de toate celelalte. Devine eroare de redefinire în momentul în care îl incluzi de două ori. (M5 în runda 1.)

**C4 —** `algortimi-utili.h:1` : **garda are o cratimă în nume.**

```
warning: extra tokens at end of #ifndef directive
```

`ALGORTIMI-UTILI_H_INCLUDED` nu e un identificator valid — cratima e operatorul minus. Garda nu protejează nimic. Folosește `ALGORTIMI_UTILI_H_INCLUDED` . (Și numele fișierului e scris greșit: „algortimi”).

**C5 —** `temaVar1.h:83` : `int A[100][100]` pentru `n,m ≤ 10` ; 40.000 de octeți pe stivă. (M2 în runda 1.) În `temaVar3.h:155` ai scris `a[25][25]` pentru `n ≤ 24` — acolo ai dimensionat exact. Aplică același lucru și aici.

**C6 —** `strlen()` în condiția buclei, recalculat la fiecare pas: `temaVar2.h:67` , `temaVar6.h:82` , `temaVar7.h:46` . (M3 în runda 1, acum în trei locuri.)

**C7 —** `temaVar1.h:62` : `minim` are o ramură în plus. (C1 în runda 1.) `return a < b ? a : b;`

**C8 —** `temaVar1.h:148` : citești din `bac1.txt` , enunțul cere `bac.txt` . (C2 în runda 1.) Iar `temaVar1.h:86` citește `m` înaintea lui `n` , invers față de enunț. (C3 în runda 1.)

**C9 —** `bin/` și `obj/` sunt în continuare urmărite de git, `.gitignore` tot nu există. (C4 în runda 1 — vezi și B1: e aceeași cauză.)

**C10 —** `coreactareVar2.h` conține doar un enunț, niciun rând de cod. Ori îl rezolvi, ori îl ștergi: e deja rezolvat în `temaVar2.h` .

**C11 —** commit-urile sunt semnate `KERNEL0S-PC\Administrator` .

```
git config --global user.name "Andrei <numele tău>"
git config --global user.email "dariusswts@gmail.com"
```

## Before / After — B6

```
// ÎNAINTE – afișează "baAcaAlaAuUreEaAt" + gunoi
for (int i = 0; i < strlen(cuvant); i++) {
    cuvantNou[poz]=cuvant[i];
    poz++;
    if (esteVocala(cuvant[i])) {
        cuvantNou[poz]=cuvant[i]-32;
        poz++;
    }
}
cout<<cuvantNou<<endl;
```

```
// DUPĂ – afișează exact "baAcaAlaAuUreEaAt"
int n = strlen(cuvant);
for (int i = 0; i < n; i++) {
    cuvantNou[poz++] = cuvant[i];
    if (esteVocala(cuvant[i]))
        cuvantNou[poz++] = cuvant[i] - 32;
}
cuvantNou[poz] = '\0';
cout << cuvantNou << endl;
```

---

## Q&A — trei întrebări

1. `temaVar3.h` are `cif` definit de două ori, iar a doua definiție e corectă. De ce refuză compilatorul un fișier în care ambele funcții sunt bune? Ce ar trebui să aleagă, și pe baza cărei informații?
  2. În `temaVar5.h`, `f(3,17)` afișează `7*****`. Cifra `7` vine din apelul `f(8,7)`, care e ultimul apel făcut — și totuși se afișează **prima**. Explică de ce, folosind cuvântul „stivă”.
  3. Ai șters `buletinAnalyze_2010.pdf` din folder și ai făcut commit. Poate cineva să-l mai descarce? Dacă da, de unde anume?
-

## Ce urmează

Ordinea:

1. **B1** — fă repo-ul privat. Acum, înainte de orice altceva. Apoi scrie-mi.
2. **B2** — cele șase linii cu ghilimele. Include toate fișierele în `main.cpp` și compilează. Zece minute, și abia după asta știi pe ce stai.
3. **B3, B4** — duplicatele. Un nume, un fișier.
4. **B7** — `temaVar5.h` : pasează matricea ca parametru, cum faci deja în `temaVar3.h` .
5. **B5, B6, B8** — trei reparații mici, cu efect vizibil la rulare.

Ce e de reținut din runda asta: ai rezolvat nouă exerciții noi și **toate nouă sunt corecte**. Gândirea e în regulă. Dar ai scris 1137 de linii peste un proiect care nu compila, fără să compilezi nici măcar o dată — și de aia patru fișiere din zece sunt azi moarte, iar unul dintre ele ( `temaVar5.h` ) a fost scris **după** ce ți-am spus exact care e greșeala.

Poarta de compilare nu e o formalitate de la finalul lucrului. E singurul lucru care îți spune dacă ceea ce ai scris există cu adevărat.

---

## Runda 1 — 2026-09-15 — sha 19f5193

Branch `master` , 4 fișiere, 500 de linii. `temaVar3.h` și `temaVar4.h` nu compilau.

-  **B1** — ghilimele tipografice + bara `|` din foaia de examen, în `temaVar3.h:63,67` și `temaVar4.h:82,87` · **NEATINS** (runda 2, B2)
-  **B2** — `rezolvare5` sărea consoanele și nu dubla vocala · **ÎNCHIS** (logica e corectă acum; a rămas terminatorul de șir — runda 2, B6)
-  **B3** — `varianta-1.h:10` , condiția gândită corect dar bifată litera `b` în loc de `d` · **NEATINS** (runda 2, C2)
-  **B4** — șapte cerințe fără rezolvare · **2 din 7 închise** (numerele divizibile cu n din `bac.txt`  rulat, și numerele cu cel mult 2 cifre din `NR.TXT`  ); una rezolvată greșit (matricea cu diagonala secundară — runda 2, B5); patru rămân (runda 2, C1)
-  **M1** — `temaVar4.h` fără `#include <iostream>` · **NEATINS**
-  **M2** — `int A[100][100]` pentru `n,m ≤ 10` · **NEATINS** (runda 2, C5)
-  **M3** — `strlen()` recalculat în condiția buclei · **NEATINS**, acum în 3 locuri (runda 2, C6)
-  **M4** — `main.cpp` nu apelează nimic · **NEATINS**, acum `main()` e complet gol (runda 2, M1)

- **M5** — `varianta-1.h` fără gardă de includere · **NEATINS** (fișierul se numește acum `temaVar1.h` — runda 2, C3)
- **C1** — `minim` cu o ramură în plus · **NEATINS** (runda 2, C7)
- **C2** — `bac1.txt` în loc de `bac.txt` · **NEATINS** (runda 2, C8)
- **C3** — citește `m` înaintea lui `n` · **NEATINS** (runda 2, C8)
- **C4** — `bin/` și `obj/` comise, fără `.gitignore` · **NEATINS** (runda 2, C9 — și cauza lui B1)

**Verificate prin rulare și corecte în runda 1:** `f(121,1)` → `12345` , `f(12345)` → `5310135` , `f(26)` → `xyyy` , matricea `min(i,j)` , mijlocul segmentului, `cif(a,b)` , și itemii cu variante de răspuns de la variantele 2, 3 și 4.



**MyCodeSchool**

Material didactic MyCodeSchool

[mycodeschool.ro](https://mycodeschool.ro)